Machine Learning Basics Lecture 4: SVM I

Princeton University COS 495

Instructor: Yingyu Liang

Review: machine learning basics

Math formulation

- Given training data $\{(x_i, y_i): 1 \leq i \leq n\}$ i.i.d. from distribution D
- Find $y = f(x) \in \mathcal{H}$ that minimizes $\hat{L}(f) = \frac{1}{n}\sum_{i=1}^{n} l(f, x_i, y_i)$
- s.t. the expected loss is small

$$L(f) = \mathbb{E}_{(x,y)\sim D}[l(f, x, y)]$$

Machine learning

- Collect data and extract features
- Build model: choose hypothesis class $\mathcal{H}$ and loss function l
- Optimization: minimize the empirical loss

Loss function

- l_2 loss: linear regression
- Cross-entropy: logistic regression

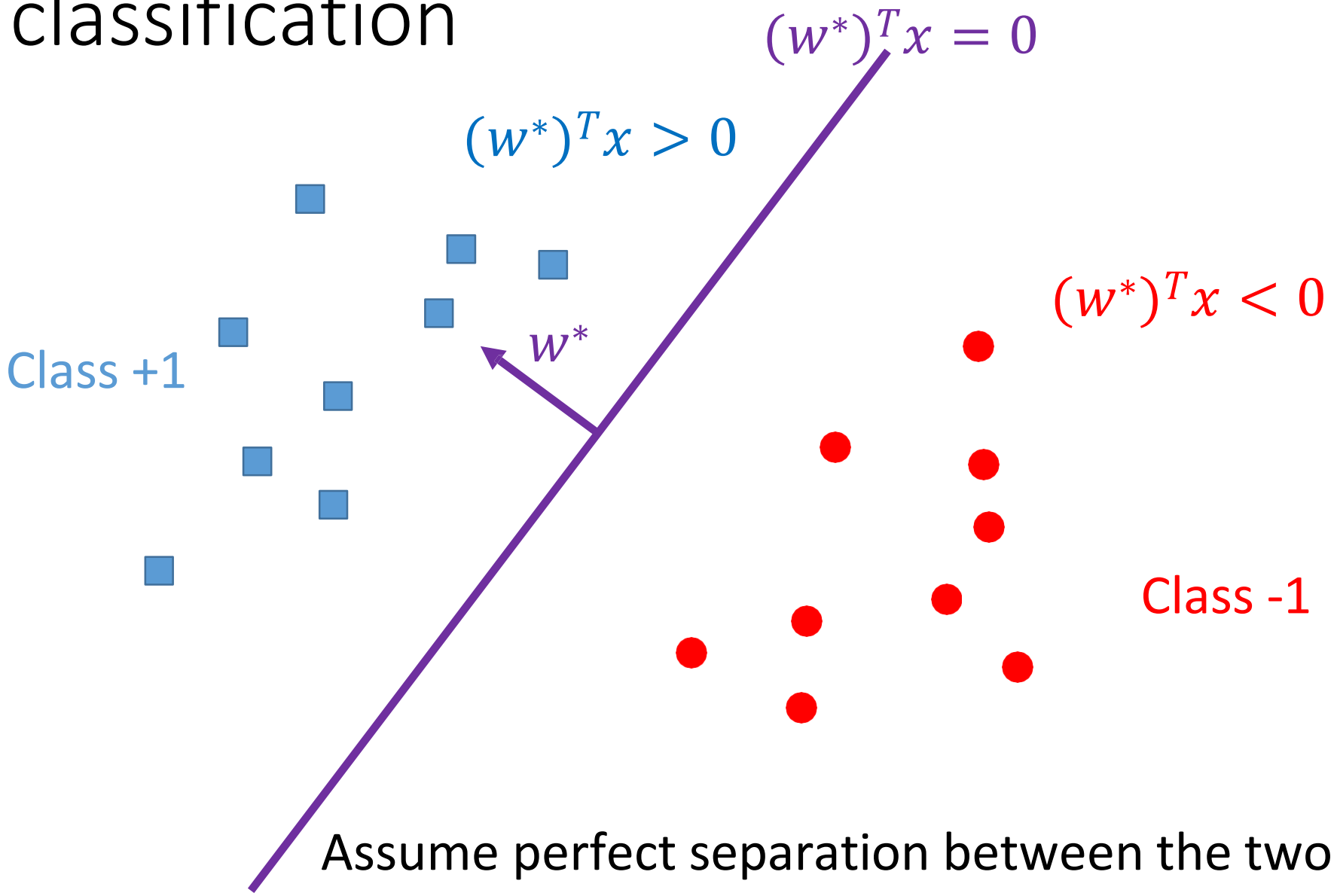
- General principle: maximum likelihood estimation (MLE)
  - l_2 loss: corresponds to Normal distribution
  - logistic regression: corresponds to sigmoid conditional distribution

Optimization

- Linear regression: closed form solution
- Logistic regression: gradient descent
- Perceptron: stochastic gradient descent

- General principle: local improvement
  - SGD: Perceptron; can also be applied to linear regression/logistic regression

Linear classification



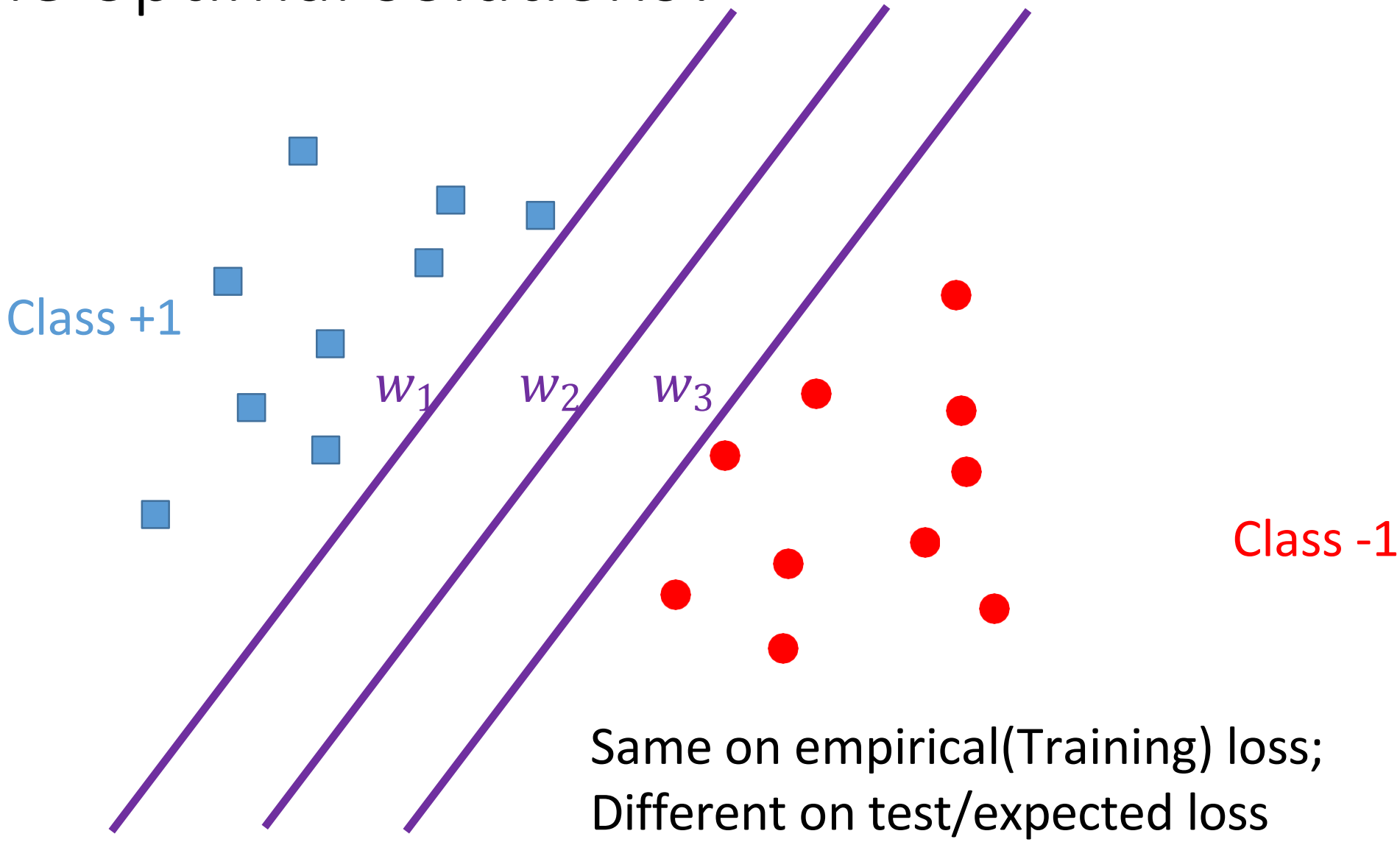
Assume perfect separation between the two classes

Attempt

- Given training data $\{(x_i, y_i): 1 \leq i \leq n\}$ i.i.d. from distribution D
- Hypothesis $y = \text{sign}(f_w(x)) = \text{sign}(w^T x)$
  - $y = +1$ if $w^T x > 0$
  - $y = -1$ if $w^T x < 0$

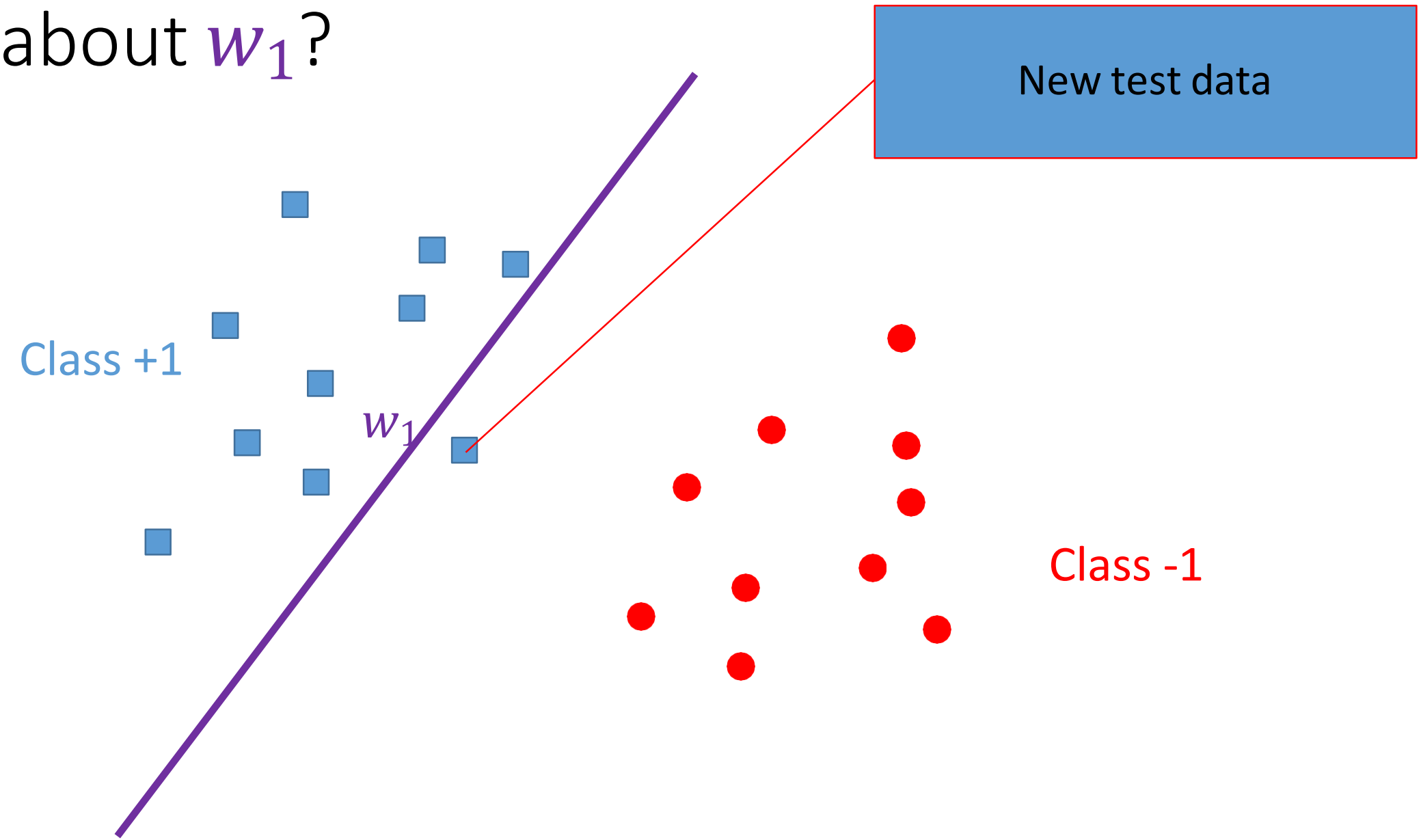
- Let's assume that we can optimize to find w

Multiple optimal solutions?

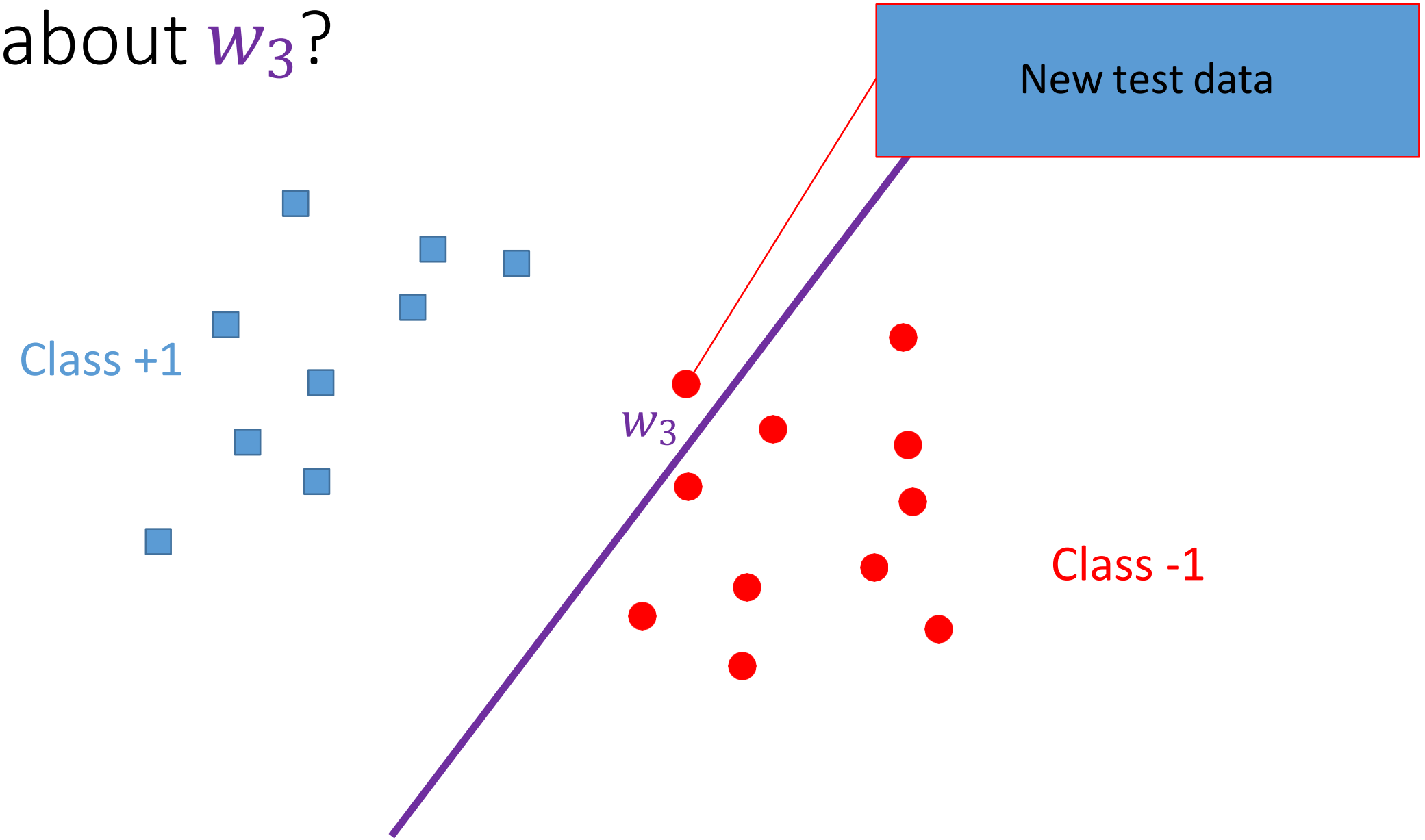


Same on empirical(Training) loss;
Different on test/expected loss

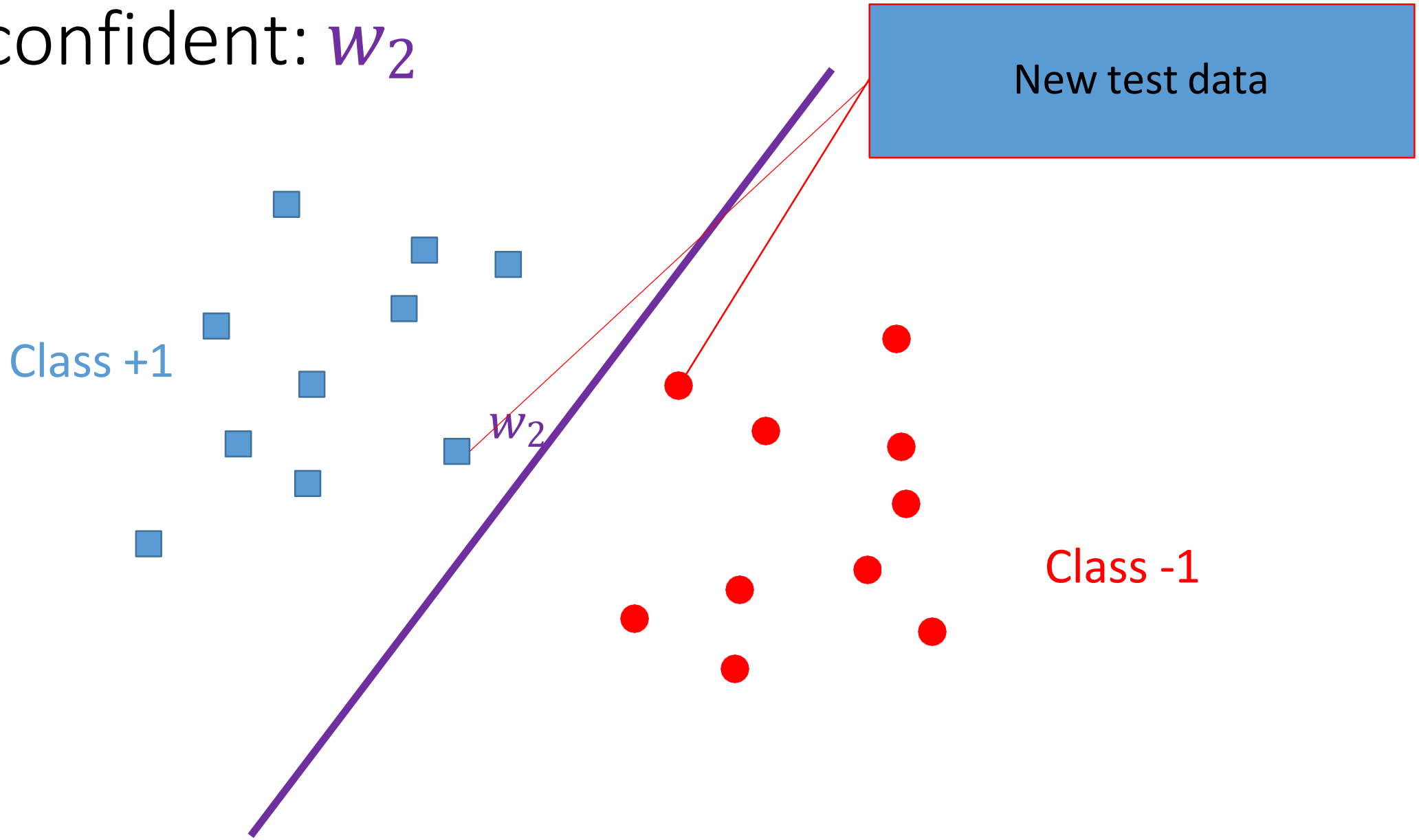
What about w_1?



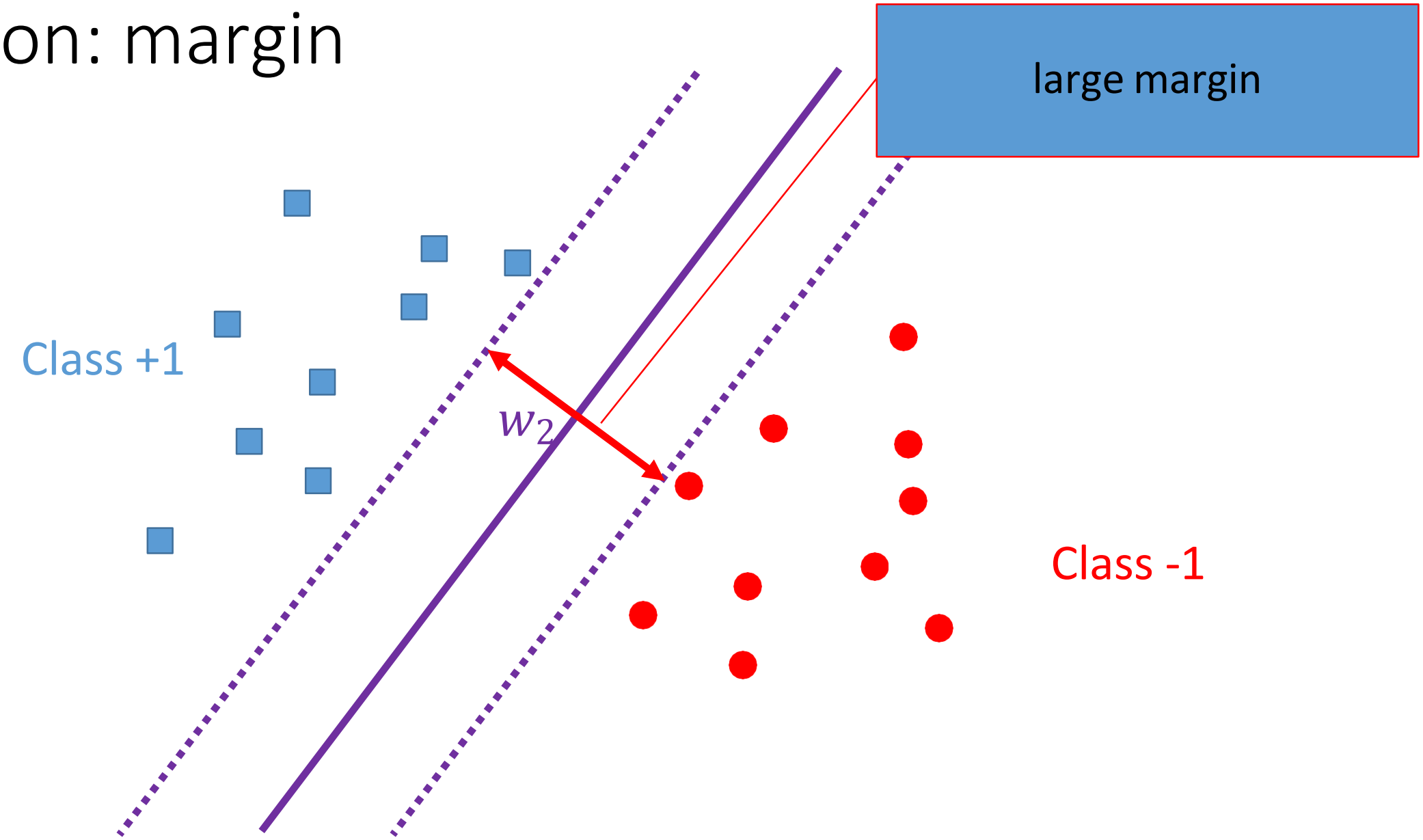
What about w_3?



Most confident: w_2



Intuition: margin



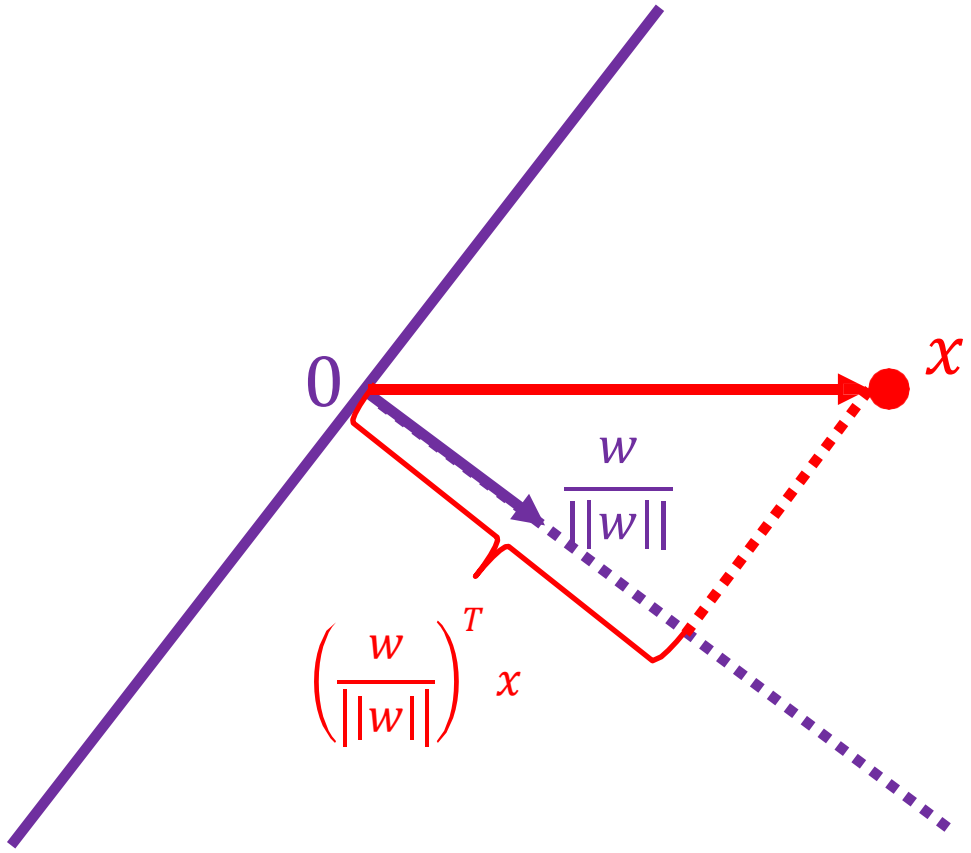
Margin: The concept of margin aims to find a hyperplane that maximizes this margin to improve classification performance and generalization ability.

Margin

- Lemma 1: x has distance $\frac{|f_w(x)|}{||w||}$ to the hyperplane $f_w(x) = w^T x = 0$

Proof:

- w is orthogonal to the hyperplane
- The unit direction is $\frac{w}{||w||}$
- The projection of x is $\left(\frac{w}{||w||}\right)^T x = \frac{f_w(x)}{||w||}$



Margin: with bias

- Claim 1: w is orthogonal to the hyperplane $f_{w,b}(x) = w^T x + b = 0$

Proof:

- pick any x_1 and x_2 on the hyperplane
- $w^T x_1 + b = 0$
- $w^T x_2 + b = 0$

- So $w^T(x_1 - x_2) = 0$

Margin: with bias

- Claim 2: 0 has distance $\frac{-b}{||w||}$ to the hyperplane $w^T x + b = 0$

Proof:

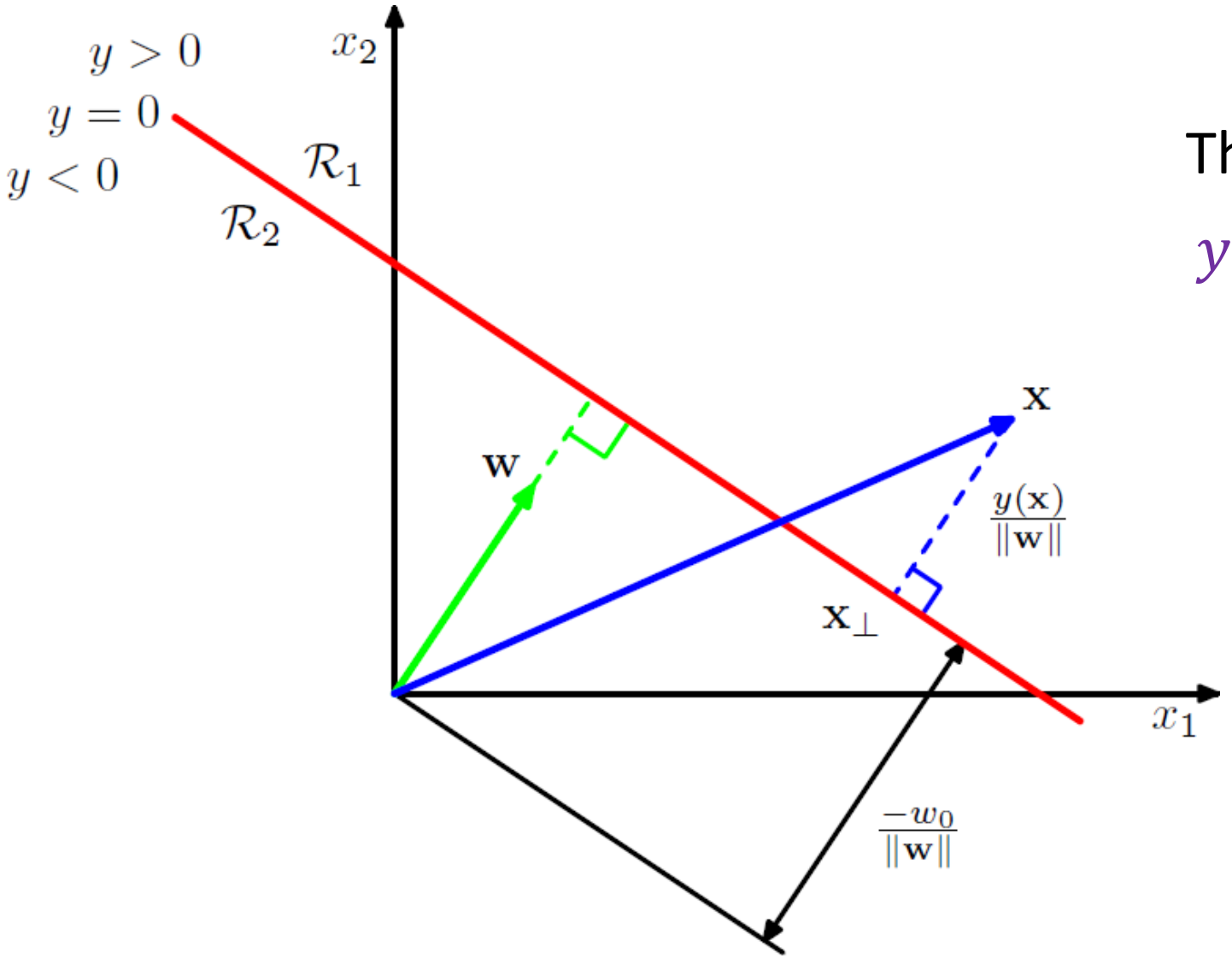
- Pick any x_1 the hyperplane
- Project x_1 to the unit direction $\frac{w}{||w||}$ to get the distance
- $\left(\frac{w}{||w||}\right)^T x_1 = \frac{-b}{||w||}$ since $w^T x_1 + b = 0$

Margin: with bias

- Lemma 2: x has distance $\frac{|f_{w,b}(x)|}{||w||}$ to the hyperplane $f_{w,b}(x) = w^T x + b = 0$

Proof:

- Let $x = x_\perp + r\frac{w}{||w||}$, then $|r|$ is the distance
- Multiply both sides by w^T and add b
- Left hand side: $w^T x + b = f_{w,b}(x)$
- Right hand side: $w^T x_\perp + r\frac{w^T w}{||w||} + b = 0 + r||w||$



The notation here is:

$$y(x) = w^T x + w_0$$

Figure from *Pattern Recognition and Machine Learning*, Bishop

Functional Margin

Lets formalize the notions of the functional Given a training example $(x^{(i)}, y^{(i)})$, we define the **functional margin** of (w, b) with respect to the training example

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x + b).$$

Note that if $y^{(i)} = 1$, then for the functional margin to be large (i.e., for our prediction to be confident and correct), then we need $w^T x + b$ to be a large positive number. Conversely, if $y^{(i)} = -1$, then for the functional margin to be large, then we need $w^T x + b$ to be a large negative number. Moreover, if $y^{(i)}(w^T x + b) > 0$, then our prediction on this example is correct. (Check this yourself.) Hence, a large functional margin represents a confident and a correct prediction.

Geometric Margin

It gives actual distance

$$\gamma^{(i)} = \frac{w^T x^{(i)} + b}{||w||}$$

Support Vector Machine (SVM)

SVM: objective

- Margin over all training data points:

$$\gamma = \min_i \frac{|f_{w,b}(x_i)|}{||w||}$$

- Since only want correct $f_{w,b}$, and recall $y_i \in \{+1, -1\}$, we have

$$\gamma = \min_i \frac{y_i f_{w,b}(x_i)}{||w||}$$

- If $f_{w,b}$ incorrect on some x_i, the margin is negative

SVM: objective

- Maximize margin over all training data points:

$$\max_{w,b} \gamma = \max_{w,b} \min_{i} \frac{y_i f_{w,b}(x_i)}{||w||} = \max_{w,b} \min_{i} \frac{y_i (w^T x_i + b)}{||w||}$$

- A bit complicated …

SVM: simplified objective

- Observation: when (w, b) scaled by a factor c, the margin unchanged

$$\frac{y_i(cw^T x_i + cb)}{||cw||} = \frac{y_i(w^T x_i + b)}{||w||}$$

- Let's consider a fixed scale such that

$$y_{i^*}(w^T x_{i^*} + b) = 1$$

where x_{i^*} is the point closest to the hyperplane

SVM: simplified objective

- Let's consider a fixed scale such that

$$y_{i^*}(w^T x_{i^*} + b) = 1$$

  where x_{i^*} is the point closet to the hyperplane
- Now we have for all data

$$y_i(w^T x_i + b) \geq 1$$

  and at least for one i the equality holds
- Then the margin is $\frac{1}{||w||}$

SVM: simplified objective

- Optimization simplified to

$$\min_{w,b} \frac{1}{2}\left|\left|w\right|\right|^2$$

$$y_i(w^T x_i + b) \geq 1, \forall i$$

- How to find the optimum W^*?

SVM: optimization

- Optimization (Quadratic Programming):

$$\min_{w,b} \frac{1}{2}||w||^2$$

→ This is constrained optimization Problem

$$y_i(w^T x_i + b) \geq 1, \forall i$$

It can be converted to unconstraint Optimization Problem using Lagrangian Multiplier that gives us following expression: L = f(x) - ∝g(x)

- Solved by Lagrange multiplier method:

$$\mathcal{L}(w, b, \boldsymbol{\alpha}) = \frac{1}{2}||w||^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$$

where $\boldsymbol{\alpha}$ is the Lagrange multiplier

Lagrange multiplier

Lagrangian

- Consider optimization problem:

$$\min_{w} f(w)$$

$$h_i(w) = 0, \forall 1 \leq i \leq l$$

- Lagrangian:

$$\mathcal{L}(w, \boldsymbol{\beta}) = f(w) + \sum_{i} \beta_i h_i(w)$$

where β_i's are called Lagrange multipliers

Lagrangian

- Consider optimization problem:

$$\min_{w} \; f(w)$$

$$h_i(w) = 0, \forall 1 \leq i \leq l$$

- Solved by setting derivatives of Lagrangian to 0

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0; \quad \frac{\partial \mathcal{L}}{\partial \beta_i} = 0$$

Lagrangian Expression

$$w = \Sigma \alpha_i y_i x_i \quad and \quad \Sigma \alpha_i y_i = 0$$

$$L(\omega, b) = \frac{1}{2}(w \cdot w) - \Sigma \alpha_i y_i b - \Sigma \alpha_i y_i . w . xi + \Sigma \alpha_i$$

$$\rightarrow \frac{1}{2}(w \cdot w) = \frac{1}{2} \Sigma \alpha_i . \alpha_j . y_i . y_j . (x_i . x_j)$$

$$\rightarrow \Sigma \alpha_i y_i . w . xi = \Sigma \alpha_i \, \alpha_j . yi . yj . (x_i . xj)$$

$$L = \tfrac{1}{2} \Sigma \alpha_i . \alpha_j . y_i . y_j . (x_i . x_j) - \Sigma \alpha_i \, \alpha_j . yi . yj . (x_i . xj) + \Sigma \alpha_i$$

$$L = \Sigma \alpha_i - \frac{1}{2} \Sigma \alpha_i \, \alpha_j . yi . yj . (x_i . xj) \qquad \boldsymbol{\alpha_i \geq 0}$$

If alpha is zero then data is not support vector , if alpha is high then it will be support vector, if alpha is extremely high then it will be outlier.

Classify unknown Data z:

D(z) = sgn($\Sigma\alpha_j . yj . (x_j . z) + b$)

$$b = y_i - \sum_j \alpha_j y_j (x_j \cdot x_i)$$

- b =- $\frac{1}{2}\min\left(\Sigma\alpha_i . yi . (x_i . x_j)\right) \forall y_i = +1$
  $+ \max(\Sigma\alpha_i . yi . (x_i . xj)) \forall\ y_i = -1$

So, SVM tells where hyperplane should be placed, that makes classifier more robust and generalized

Classify unknown Data z:

D(z) = sgn($\Sigma\alpha_j . yj . (x_j . z) + b$)

$$b = y_i - \sum_j \alpha_j y_j (x_j \cdot x_i)$$

- b =- $\frac{1}{2}\min\left(\Sigma\alpha_i . yi . (x_i . x_j)\right) \forall y_i = +1$
  $+ \max(\Sigma\alpha_i . yi . (x_i . xj)) \forall\ y_i = -1$

So, SVM tells where hyperplane should be placed, that makes classifier more robust and generalized

Kernel methods

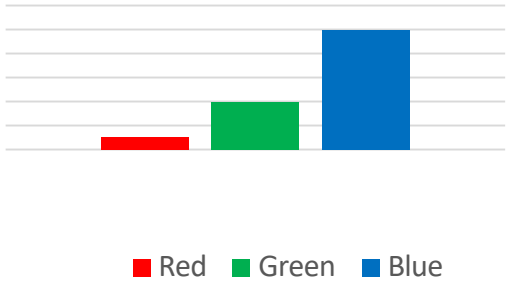
Features

x

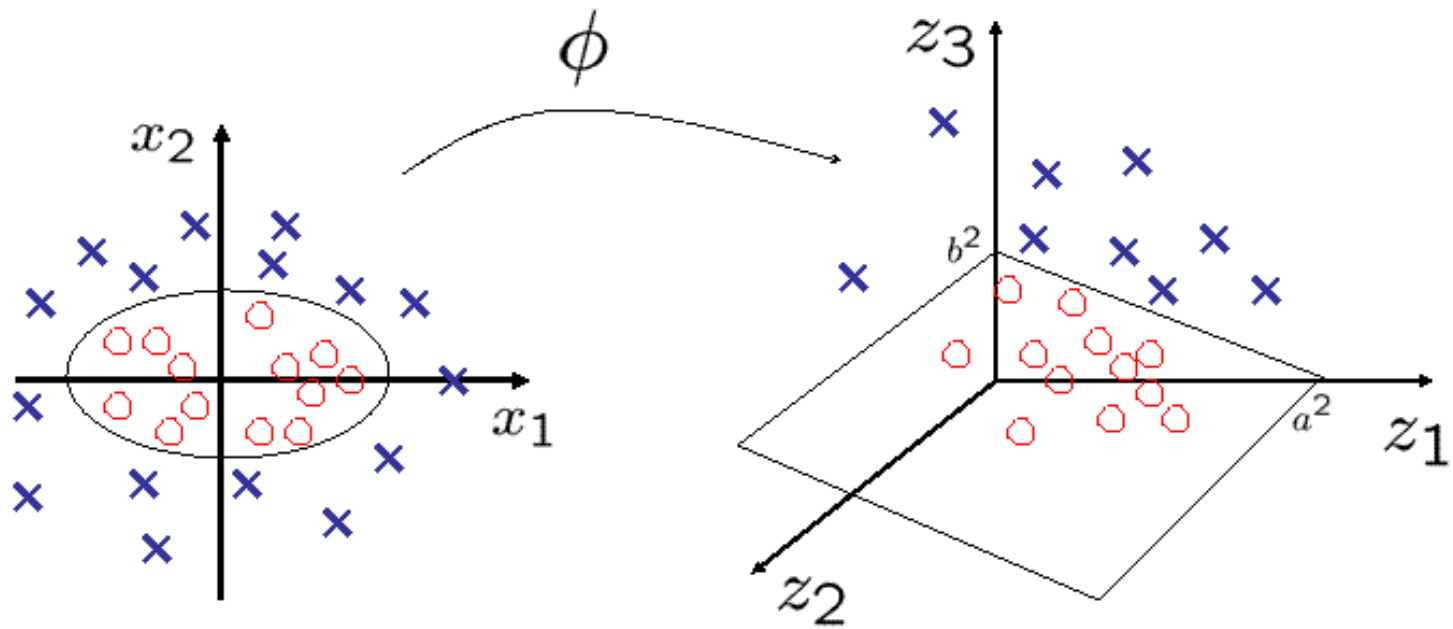
Extract features

$\phi(x)$

Color Histogram



Features



$$\phi : (x_1, x_2) \longrightarrow (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

$$\left(\frac{x_1}{a}\right)^2 + \left(\frac{x_2}{b}\right)^2 = 1 \longrightarrow \frac{z_1}{a^2} + \frac{z_3}{b^2} = 1$$

Features

- Proper feature mapping can make non-linear to linear
- Using SVM on the feature space $\{\phi(x_i)\}$: only need $\phi(x_i)^T\phi(x_j)$

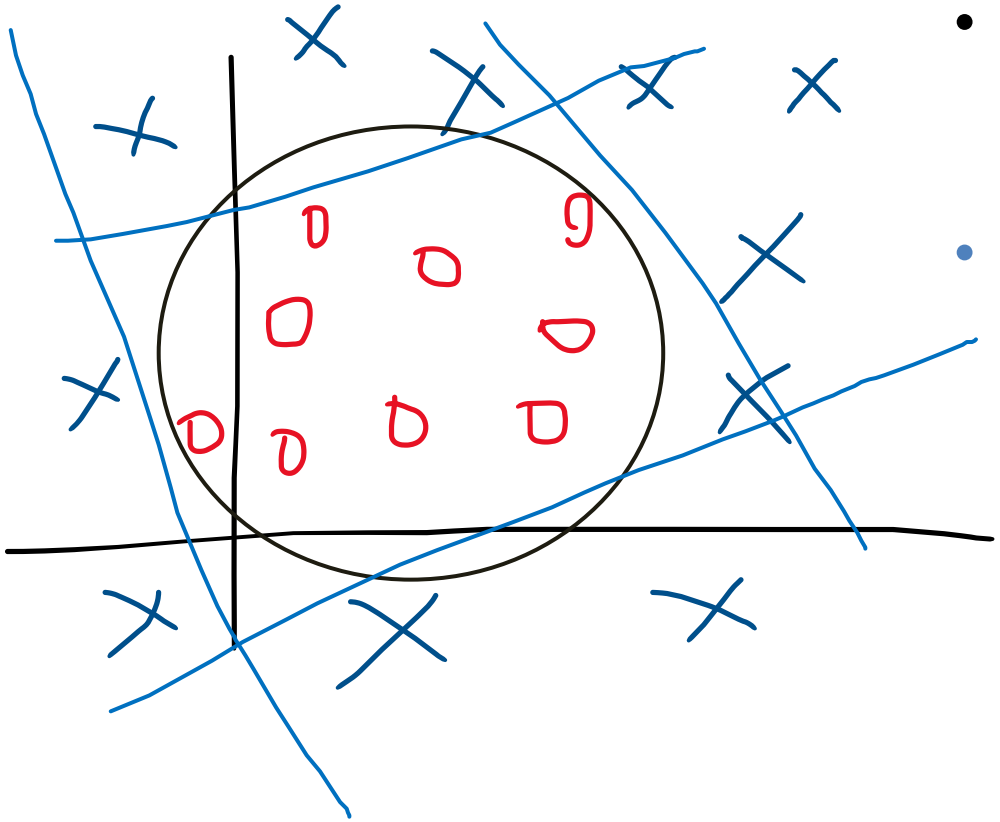
- Conclusion: no need to design $\phi(\cdot)$, only need to design

$$k(x_i, x_j) = \phi(x_i)^T\phi(x_j)$$

Linear SVMs: Overview

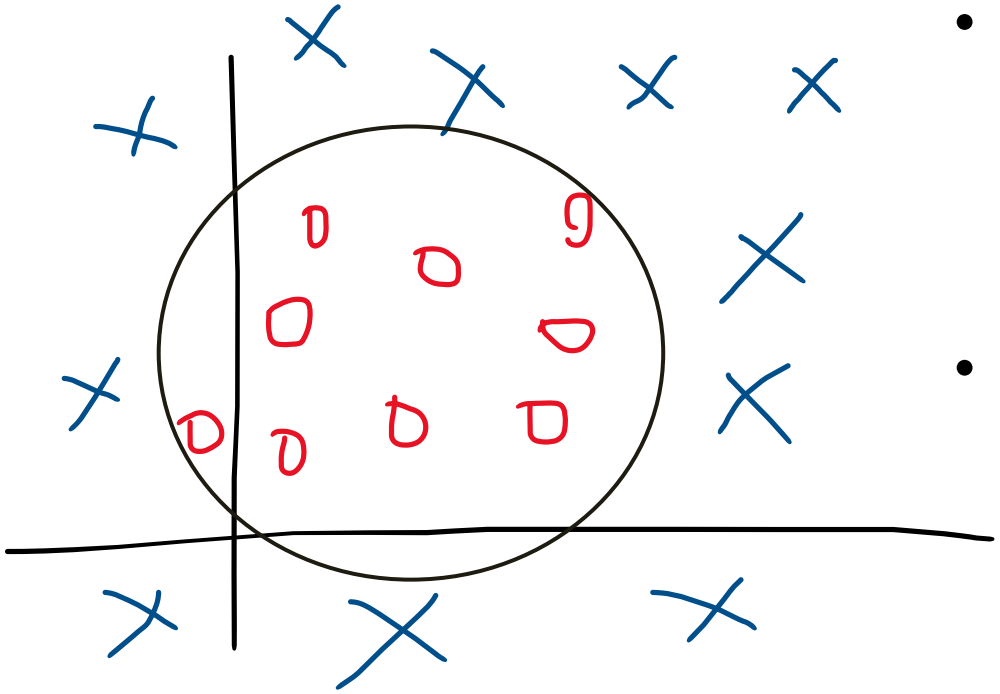
The classifier is a non-probabilistic and linear classifier.
It is an Optimal Margin Classifier.
Most “important” training points are support vectors; they define the hyperplane.
Lagrangian Multipliers: ways to convert a constrained optimization problem to one that is easier to solve
Kernels: Projecting data to higher dimensional space makes it linearly separable
It is among the best performer for Text and genomic data
It can be applied to complex datatypes beyond feature vectors i.e. graphs, sequences and relational data.

Non-Linear Classification



- A kernel is a function that is equal to an inner product in some feature space.
- A kernel tricks avoids explicit mapping to get linear algorithms to learn a nonlinear function or decision boundary
  - Linear Kernel $K(x_i,x_j)=x_i.x_j$
  - Polynomial Kernel $K(x_i,x_j)=(1+x_i.x_j)^p$
  - Gaussian Kernel = $e^{\frac{||xi-xj||2}{2\sigma^2}}$

Non-Linear Classification



- If data is not linearly separable, we have to place them into high dimensional space. And Compute the **L =** $\Sigma \alpha_j . yj . (x_j . z) + b$
- Replace the value of dot-product with kernel

Kernel Trick: To run SVM in high Dimensional F.V

Write algorithm in terms of (x,z)

Let there be some mapping from x_{2D} ->$\phi(x)_{nD}$

Find a way to compute K(x,z) = $\phi(x)^T\phi(z)$

Replace (x,z) in an algorithm with K(x,z)

Polynomial kernels

- Fix degree d and constant c:

$$k(x, x') = (x^T x' + c)^d$$

- What are $\phi(x)$?
- Expand the expression to get $\phi(x)$

Polynomial kernels

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^2, \quad K(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + c)^2 = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \\ \sqrt{2c}\, x_1 \\ \sqrt{2c}\, x_2 \\ c \end{bmatrix} \cdot \begin{bmatrix} x_1'^2 \\ x_2'^2 \\ \sqrt{2}\, x_1' x_2' \\ \sqrt{2c}\, x_1' \\ \sqrt{2c}\, x_2' \\ c \end{bmatrix}$$

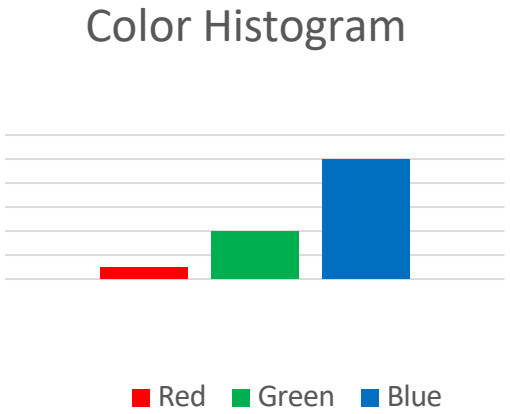
Figure from Foundations of Machine Learning, by M. Mohri, A. Rostamizadeh, and A. Talwalkar

Kernels v.s. Neural networks

Features

x

Extract features

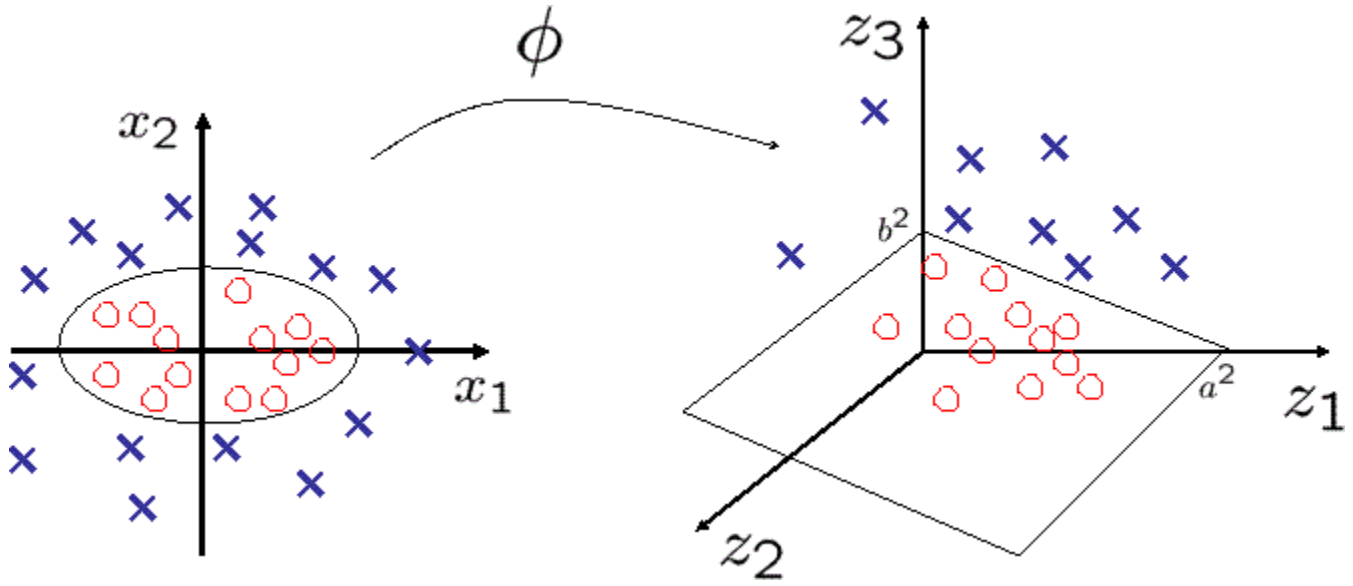


build hypothesis

$$y = w^T \phi(x)$$

Features: part of the model

Nonlinear model



build hypothesis

$$y = w^T \phi(x)$$

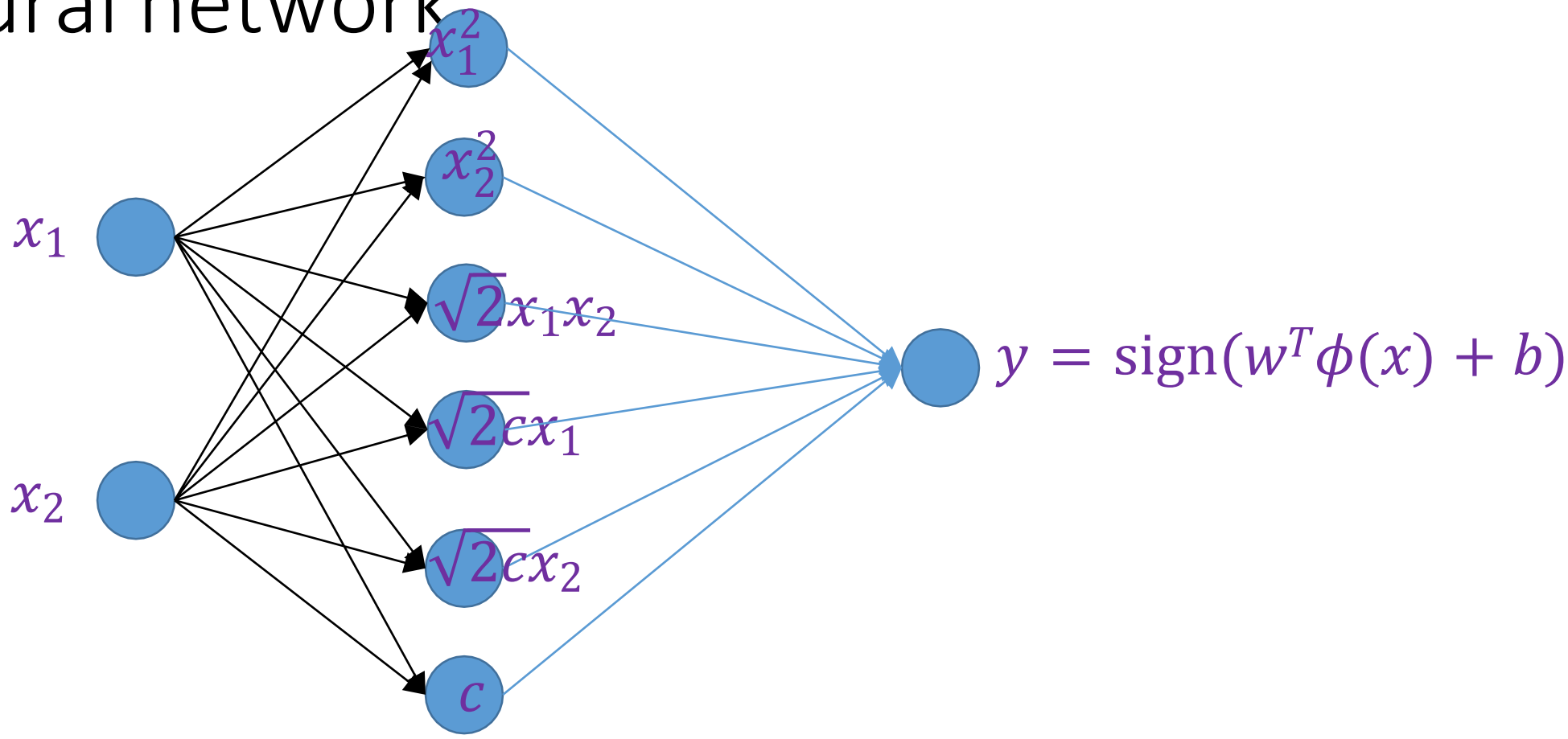
Linear model

Polynomial kernels

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^2, \quad K(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + c)^2 = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \\ \sqrt{2c}\, x_1 \\ \sqrt{2c}\, x_2 \\ c \end{bmatrix} \cdot \begin{bmatrix} x_1'^2 \\ x_2'^2 \\ \sqrt{2}\, x_1' x_2' \\ \sqrt{2c}\, x_1' \\ \sqrt{2c}\, x_2' \\ c \end{bmatrix}$$

Figure from Foundations of Machine Learning, by M. Mohri, A. Rostamizadeh, and A. Talwalkar

Polynomial kernel SVM as two layer neural network



First layer is fixed. If also learn first layer, it becomes two layer neural network

Reading

- Review Lagrange multiplier method
- E.g. Section 5 in Andrew Ng's note on SVM
  - https://web.cs.hacettepe.edu.tr/~erkut/ain311.f24/readings/cs229-notes3.pdf
    http://www.cs.princeton.edu/courses/archive/spring16/cos495/